Procedural World Generation Using Natural Language Processing

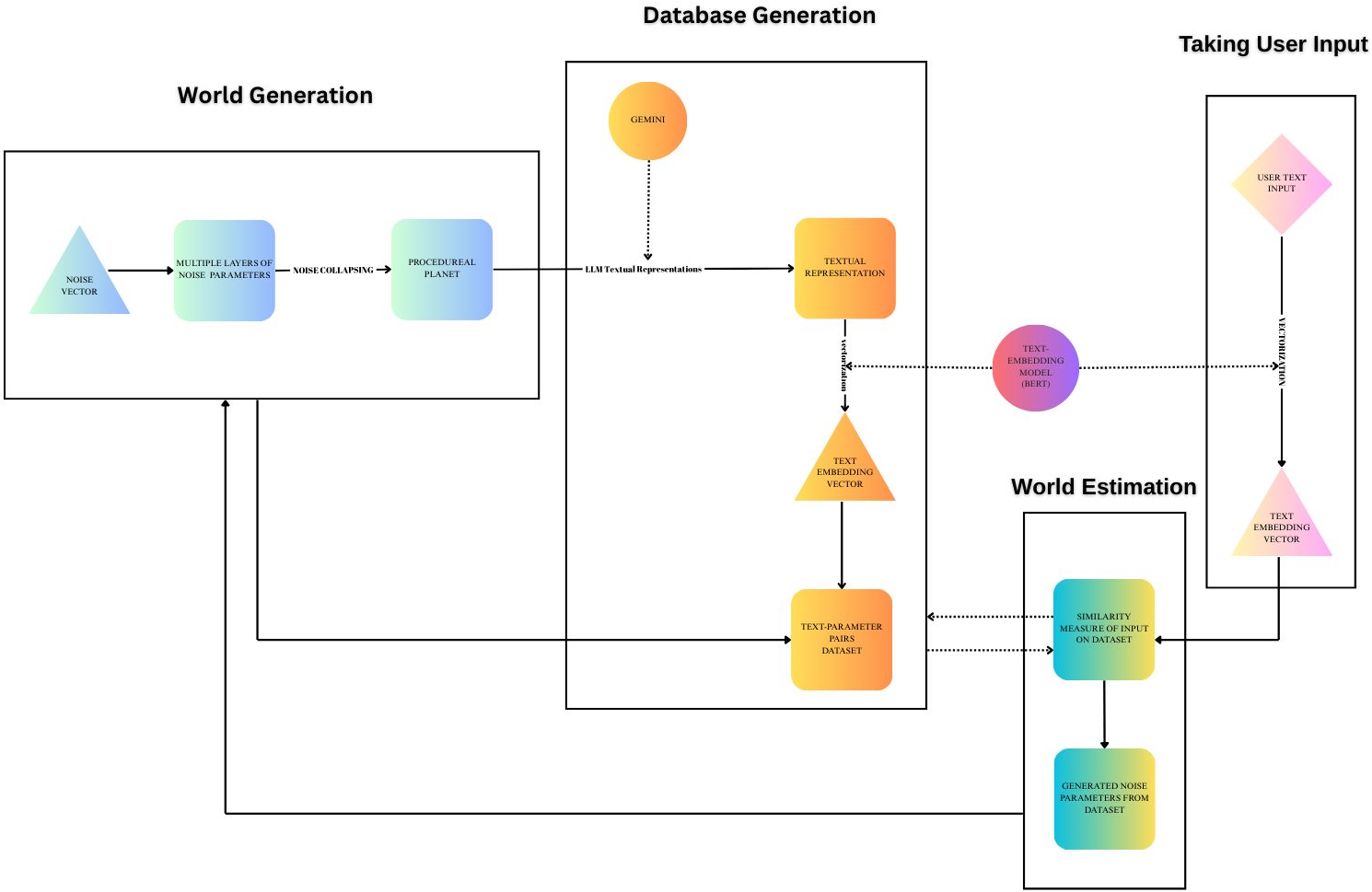
Hacettepe University Engineering Faculty
Computer Engineering Department

Advisor: Assoc. Prof. Dr. Ali Seydi Keçeli

Ahmet Yiğitalp Demirci 2210356093
Zafer İçen 2200356076
Mehmet Erdem Gündüz 2210356012
Gülşen Ebrar Cırık 2210356047

What Did We Build?

We built a system that converts user-written descriptions into visually explorable 3D worlds. The system processes each sentence by extracting important keywords and comparing them to a database of pre-generated planets. Using a custom scoring algorithm or a k-NN-based generation method, it either selects the closest existing world or directly generates a matching configuration. The result is rendered in real time as a 3D terrain, allowing users to walk through their imagined environment. The system supports context-aware session updates, enabling users to guide the output through natural language alone.


Database Generation
Taking User Input
World Generation
GEMINI
USER TEXT INPUT
NOISE VECTOR
MULTIPLE LAYERS OF NOISE PARAMETERS
NOISE COLLAPSING
PROCEDUREAL PLANET
LLM Textual Representations
TEXTUAL REPRESENTATION
VECTORIZATION
TEXT-EMBEDDING MODEL (BERT)
VECTORIZATION
TEXT EMBEDDING VECTOR
World Estimation
TEXT EMBEDDING VECTOR
TEXT-PARAMETER PAIRS DATASET
SIMILARITY MEASURE OF INPUT ON DATASET
GENERATED NOISE PARAMETERS FROM DATASET

Why Are We Doing This?

This project was born from the desire to move away from generic generative AI and instead explore whether procedural systems can be predicted and guided using structured logic. Rather than relying on opaque models, we aimed to build a transparent, controllable pipeline that translates natural language into procedural landscapes. The core idea is that if humans can anticipate the outcome of noise-based systems with enough understanding, AI might be able to do the same.

Natural language is key here—it allows users to define complex worlds without manually adjusting dozens of parameters. By making procedural generation accessible through descriptive input, we remove the need for expert knowledge and enable intuitive control over otherwise complex systems. While our initial assumptions were ambitious, the project evolved into a focused attempt to classify and predict procedural behavior—a challenge that remains central in applications from game development to geological simulation.

Our Main Goal

Our main goal is to make procedural systems controllable without sacrificing their generative power. Procedural methods offer scale and diversity, but often leave users uncertain about what will be produced. This project aims to bridge that gap—preserving randomness while enabling users to guide outcomes through natural language. By interpreting descriptive input and linking it to procedural logic, we allow users to shape the results without needing technical expertise. This proof-of-concept lays the groundwork for a general interface layer that could adapt to any procedural system with minimal configuration.

How Are We Doing It?

The system begins with a free-form sentence describing a planet. This input is processed using Gemini, which extracts structured terrain features like landmass types or water coverage. These features are compared to predefined keywords associated with our database of Perlin noise configurations. Using a custom scoring system based on simple algebra, the closest match is selected. The result is a JSON-formatted Perlin parameter vector, which is passed to a Unity WebGL rendering system that visualizes the corresponding terrain in 3D. The use of Gemini was chosen for cost-efficiency, and embedding-based similarity was abandoned due to poor separation between geography-related inputs. WebGL and JSON enabled smooth visualization and decoupled development.

Where Are We Now?

The project is currently a functional prototype with both classification and basic generation capabilities. The user interface and 3D rendering are complete, and the new k-NN-based generation method produces usable terrain outputs directly from textual descriptions. While the dataset remains small, this generative step marks a major milestone. Earlier obstacles—including unstable APIs and flawed labeling—still affect consistency, but the pipeline now demonstrates its full vision: generating procedural content from language, not just selecting it.

Why Does It Work?

The system works because it is modular, interpretable, and grounded in mathematical simplicity. Each component of the pipeline—text processing, scoring, rendering—was designed to be replaceable, allowing experimentation without destabilizing the entire system. The keyword-based scoring algorithm, while initially a placeholder, proved more effective than embedding-based approaches due to its discrete logic and resilience to noise. By avoiding over-engineering and embracing clear rules, the system provides directional accuracy in matching worlds to user intent. Even quick decisions like adopting WebGL contributed significantly by simplifying deployment and cross-platform testing.

Why Doesn’t It Work?

The system faces major limitations due to its dependence on unstable components and insufficient data. Gemini, used for text processing, frequently returns incorrect or empty responses, which has significantly slowed development. Classification results, though sometimes accurate, are inconsistent—partly due to a small and error-prone dataset of only 100 entries. Early attempts at automatic labeling with LLMs further compromised the database, producing misleading annotations. Most failures stem from unreliable input processing and low-quality data. Addressing these issues—particularly by building a verified, larger dataset and replacing or fine-tuning the text model—would resolve the majority of current problems.

How Can We Improve It?

Future improvements begin with building a larger, verified dataset to enhance classification and support generative modeling. Replacing Gemini with a more stable solution—such as a fine-tuned LLM or a task-specific classifier—would increase reliability across the pipeline. Generation can be improved using structured models like autoencoders or interpretable ML techniques. In the long term, we envision a generalized protocol for procedural systems, allowing the language-based interface to adapt across different platforms. A refined version of this project would maintain the same interface but offer more accurate, robust, and extensible functionality.

Thank You For Listening